



PPQ 5

All results can be found in the `./results_*` directories, submitted as part of the `Q5_bmjm501.tar`.

Below is the data table for part a-c on `teaching0`, noting the following information:

GCC Version: GNU Fortran (GCC) 12.3.0
Intel Version: ifort (IFORT) 2021.7.1 20221019
CPU: Intel(R) Xeon(R) Gold 5416S

Compiler Flags	Actions to source (letter refers to part and result directory)	No of inaccurate values
gfortran -O0	No changes (a)	135
ifort -O0	No changes (a)	135
gfortran -O0	Changed single to double precision (b)	82
ifort -O0	Changed single to double precision (b)	82
gfortran -O0	Removed print statement, using double precision (c1)	82
ifort -O0	Removed print statement, using double precision (c1)	82
gfortran -O3	Removed print statement, using double precision (c2)	82
ifort -O3	Removed print statement, using double precision (c2)	0
gfortran -Ofast	Removed print statement, using double precision (c3)	82

Compiler Flags	Actions to source (letter refers to part and result directory)	No of inaccurate values
ifort -Ofast	Removed print statement, using double precision (c3)	0
gfortran -O0	Removed print statement, using single precision (c4)	135
ifort -O0	Removed print statement, using single precision (c4)	135
gfortran -O3	Removed print statement, using single precision (c5)	135
ifort -O3	Removed print statement, using single precision (c5)	629
gfortran -Ofast	Removed print statement, using single precision (c6)	629
ifort -Ofast	Removed print statement, using single precision (c6)	629

Below is the data table for part a-c on **VIKING** , noting the following information:

GCC Version: GNU Fortran (GCC) 13.2.0
 Intel Version: ifort (IFORT) 2021.9.0 20230302
 CPU: AMD EPYC3 7643

Compiler Flags	Actions to source (letter refers to part and result directory)	No of inaccurate values
gfortran -O0	No changes (a)	135
ifort -O0	No changes (a)	135

Compiler Flags	Actions to source (letter refers to part and result directory)	No of inaccurate values
gfortran -O0	Changed single to double precision (b)	82
ifort -O0	Changed single to double precision (b)	82
gfortran -O0	Removed print statement, using double precision (c1)	82
ifort -O0	Removed print statement, using double precision (c1)	82
gfortran -O3	Removed print statement, using double precision (c2)	82
ifort -O3	Removed print statement, using double precision (c2)	0
gfortran -Ofast	Removed print statement, using double precision (c3)	82
ifort -Ofast	Removed print statement, using double precision (c3)	0
gfortran -O0	Removed print statement, using single precision (c4)	135
ifort -O0	Removed print statement, using single precision (c4)	135
gfortran -O3	Removed print statement, using single precision (c5)	135
ifort -O3	Removed print statement, using single precision (c5)	310
gfortran -Ofast	Removed print statement, using single precision (c6)	310

Compiler Flags	Actions to source (letter refers to part and result directory)	No of inaccurate values
ifort -Ofast	Removed print statement, using single precision (c6)	310

The tables are exactly the same until the last 3 rows.

The code for both `single` and `double` precision without any optimisation are the expected results.

There should be some inaccurate values due to the nature of the binary representation of the base 10 numbers, such that when doing things like dividing, there is a loss of precision.

When changing from `single` to `double` precision, the number of inaccurate values decrease on both computers with both compilers at `-00`. This makes sense as a `double` can store more digits accurately than a `single`, so the comparison of the two numbers results in less differences found.

Then removing the print statement does not change anything for `-00` as expected for both `single` and `doubles`. There are only two major deviations from expected results.

The first is the same across both computers where the `ifort` compilers with flag `-03` and `-Ofast` give 0 inaccurate values for `double`. This implies that the compiler was over aggressive when it came to optimising the loop, possibly changing the order of the operations and thus changing the floating point arithmetic order (recall floating point arithmetic is not associative). Notably `gfortran` did not do this. It is also worth noting that the Intel compiler on `teaching0` that has an Intel CPU had no differences when using the Intel compiler on `VIKING` with an AMD CPU.

The second is that for `single`, `teaching0` had 629 inaccuracies for aggressive optimisation in both `gfortran` and `ifort` case. On `VIKING` these differences were much lower, sitting at 310. This implies that both compiler version and flags play a huge role into how it changes the floating point arithmetic.

In general, compiler version and compiler flags can change the results of simple calculations by changing the innate floating point arithmetic too much, so that the errors accumulate differently, resulting in different answers.

Number of digits of precision

The results I got from `./main`, compiled on an AMD Ryzen 5 3500U with `g++ -O0 Q5_bmjm501.cpp -o main` were:

```
Float digits of precision: 8. Double digits of precision: 16
```

This falls within the IEEE 754 standard. The explanation of the code can be found below, or in source `Q5_bmjm501.cpp`

```

#include <iostream>

int main()
{
    /* Need a way of determining accuracy i.e. comparing
    to a number to itself plus a small change, while counting number of digits
    i.e. dividing by 10
    */

    float flt = 1.0f; // Specify 'f' to make sure it is float
    double dbl = 1.0;

    float delta_f = 1.0f; // Initial small change
    double delta_d = 1.0;

    int digits_f = 0, digits_d = 0;

    // When flt+delta_f == flt, we have lost accuracy
    // A number plus a really small number algebraically never is the same number
    // Thus is we break this condition, the computer thinks the numbers are the same
    while (flt + delta_f != flt)
    {
        delta_f /= 10.0f; // Divide by 10 as we are checking digits
        digits_f++;
    }

    // When dbl+delta_d == dbl, we have lost accuracy
    while (dbl + delta_d != dbl)
    {
        delta_d /= 10.0f; // Divide by 10 as we are checking digits
        digits_d++;
    }

    printf("Float digits of precision: %d. Double digits of precision: %d \n", digits_f, digits_d);

    return 0;
}

```